

Tugas Pemrograman Dasar TLS 2026

Dokumen Solusi Perancangan Algoritma & Logika (Fase 1)

Nama Lengkap: Kia Nur Padilah

Kelompok: Onyx

Problem 1: The Last Astronaut (Pseudocode)

ALGORITMA TheLastAstronaut

ISIAN: N (Jumlah Astronot), K (Langkah awal hitungan)

KELUARAN: Urutan astronot tereliminasi, Astronot terakhir yang bertahan

DEKLARASI:

lingkaran : array of boolean (ukuran N+1, diisi true)

sisaAstronot, indeksSekarang, hitung, astronotKe : integer

ALGORITMA:

sisaAstronot <- N

indeksSekarang <- 1

TULIS "Urutan Eliminasi:"

SELAMA sisaAstronot > 1 **LAKUKAN**

hitung <- 0

// Cari astronot ke-K yang masih dalam lingkaran

SELAMA hitung < K **LAKUKAN**

JIKA lingkaran[indeksSekarang] == true **MAKA**

 hitung <- hitung + 1

JIKA hitung < K **MAKA**

 indeksSekarang <- (indeksSekarang mod N) + 1

AKHIR JIKA

AKHIR SELAMA

// Eliminasi astronot

lingkaran[indeksSekarang] <- false

astronotKe <- indeksSekarang

TULIS astronotKe

// Update nilai K sesuai aturan

JIKA astronotKe MOD 2 == 0 **MAKA**

 K <- K + 2

MELAINKAN

 K <- K - 1

AKHIR JIKA

JIKA K < 2 **MAKA**

```
K <- 2
AKHIR JIKA

// Geser penunjuk ke posisi astronot berikutnya
ULANGI
    indeksSekarang <- (indeksSekarang mod N) + 1
SAMPAI lingkaran[indeksSekarang] == true

    sisaAstronot <- sisaAstronot - 1
AKHIR SELAMA

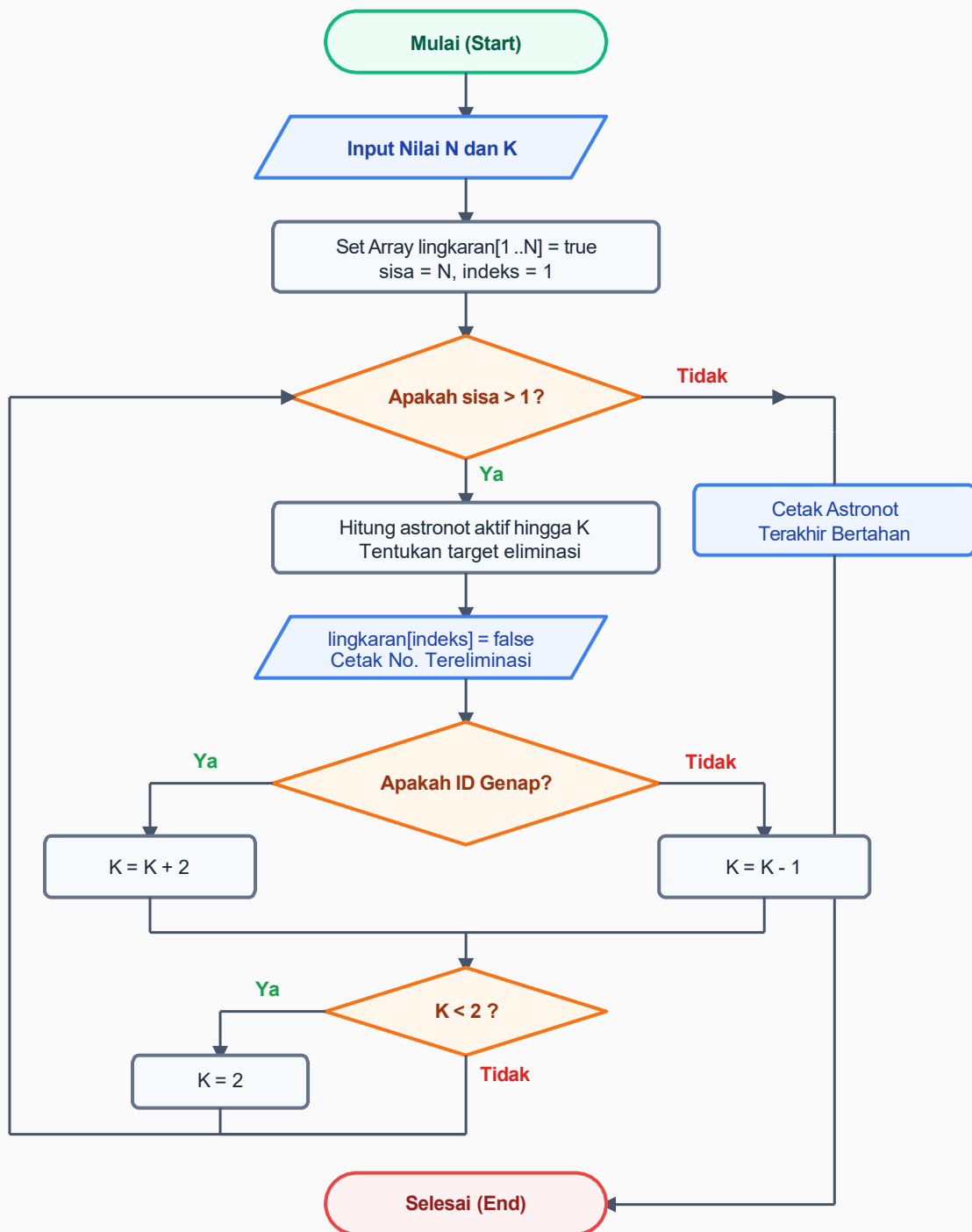
// Cari astronot terakhir yang tersisa
UNTUK i <- 1 SAMPAI N LAKUKAN
    JIKA lingkaran[i] == true MAKA
        TULIS "Astronot Terakhir: ", i
    AKHIR JIKA
AKHIR UNTUK
```

Penjelasan Logika Pemecahan Masalah

Permasalahan The Last Astronaut dirancang berdasarkan variasi klasik Josephus Problem yang melibatkan proses eliminasi melingkar secara sistematis. Pada pendekatan algoritma ini, sekumpulan astronot direpresentasikan dalam struktur array boolean berukuran N . Setiap posisi bernilai true menandakan bahwa astronot pada posisi tersebut masih aktif di dalam lingkaran. Penelusuran dilakukan berurutan menggunakan teknik aritmatika modulo untuk menjamin bahwa penunjuk indeks dapat terus berputar secara inkremental tanpa mengalami out-of-bound saat melewati posisi astronot terakhir.

Hal mendasar yang menjadi pembeda pada persoalan ini terletak pada dinamika pembaruan nilai K setelah tiap tahapan eliminasi. Setiap kali pencacah mencapai nilai K dan mengeliminasi satu astronot, sistem akan mengecek paritas (genap atau ganjil) dari nomor ID astronot yang tereliminasi. Jika nomor ID bernilai genap, nilai K melonjak sebanyak 2 unit, sedangkan jika ganjil, nilai K berkurang 1 unit. Sistem juga memuat pengkondisian batas bawah untuk memastikan bahwa nilai K tidak pernah berada di bawah 2. Proses perulangan ini terus mengeksekusi iterasi sampai sisa astronot yang bernilai true tinggal satu.

Problem 1: The Last Astronaut (Flowchart Diagram)



Problem 2: Alien-In-The-Middle (Pseudocode)

```
#include<iostream>
#include<string>
using namespace std;
string pesan,hasil="";
int main(){
    cout << "Masukkan pesan rahasia: ";
    cin>>pesan;
    for(int i=0;i<pesan.length();i++){
        if(i==0){
            hasil+=pesan[i];
        }
        else{
            int nilaiSekarang=pesan[i]-'A'+1;
            int nilaiSebelumnya=pesan[i-1]-'A'+1;
            int nilaiBaru=nilaiSekarang+nilaiSebelumnya;
            nilaiBaru=(nilaiBaru-1)%26+1;
            char hurufBaru='A'+nilaiBaru-1;
            hasil+=hurufBaru;
        }
    }
    cout<<"Pesan setelah dienkripsi: "<<hasil<<endl;
    return 0;
}
```

Penjelasan Logika Pemecahan Masalah

Permasalahan *Alien-In-The-Middle* menerapkan algoritma enkripsi berbasis pergeseran berantai (*cumulative shift cipher*). Berbeda dengan Kriptografi Caesar standar yang menggunakan kunci geser konstan, pada algoritma ini besaran pergeseran bersifat dinamis dan ditentukan oleh posisi urutan alfabet dari karakter asli yang berada tepat sebelum karakter yang sedang diproses. Huruf pertama dalam pesan tidak mengalami pergeseran (nilai penggeser awal bernilai 0).

Dalam alur pemrosesannya, setiap huruf dikonversi terlebih dahulu ke format kapital dan dihitung urutan angkanya (A=1, B=2, Z=26). Nilai baru diperoleh dengan menjumlahkan angka urutan asli dengan angka urutan karakter sebelumnya. Apabila akumulasi pergeseran melebihi angka 26, algoritma menerapkan siklus putar balik (*wrap-around*) kembali ke awal alfabet. Poin krusial dalam rancangan logika ini adalah menyimpan nilai asli huruf sebelum dienkripsi ke dalam variabel penampung kunci geser, sehingga huruf berikutnya bergeser akurat sesuai sandi yang telah disepakati.